KING MONGKUT'S UNIVERSITY OF TECHNOLOGY NORTH BANGKOK
Faculty of Applied Science / Faculty of Information Technology
FINAL EXAMINATION — SEMESTER 1, ACADEMIC YEAR 2026

Course: 060243106 & 060223119 Data Structures and Algorithms | Instructor: Dr. Pradit Pitaksatheinkul
Duration: 3 Hours (09:00 - 12:00) | Total Score: 70 Raw Marks (Scaled to 35% of Final Grade)

Student Name: ..... Student ID: ..... Section: .....

Instructions & Regulations:
1. Open Book: Textbooks, lecture handouts, and personal handwritten notes are permitted.
2. Scientific Calculator: Allowed for hash modular arithmetic and percentage calculations.
3. Strict Prohibition: Smartphones, tablets, and communication devices are strictly prohibited (Turn OFF and put in bags).
4. Answer all 7 questions clearly in the provided grids and blank lines. Points are awarded based on strict algorithmic tracing.

Question 1: Hash Table with Open Addressing and Double Hashing (10 Marks)

A hash table using Open Addressing is designed with a table capacity TableSize = 19. The system triggers automatic Rehashing when the load factor reaches or exceeds 80%.
Collision resolution is performed using Double Hashing with primary and secondary functions defined as:

- Primary Hash Function: $h_0(X) = X \bmod 19$
- Collision Offset Function: $f(i) = i \times (R - (X \bmod R))$, where R = 17 and i is the collision index (1, 2, ...)
- Probe Sequence: $h_i(X) = (h_0(X) + f(i)) \bmod 19$

Insert the following 8 keys sequentially into the hash table: 26, 45, 70, 85, 102, 123, 143, 155

Note: Each probed slot that is already occupied counts as exactly 1 collision (as explained in classroom lectures).

Slot	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
Stored Key																			

1.1 Collision Tracing Table: Write the step-by-step collision sequence for each key: **[4 Marks]**

- Key 26: h_0 = 26 mod 19 = ... → Slot: ..... (Collisions: .....)
- Key 45: h_0 = 45 mod 19 = ... → Collision at Slot: ..... → f(1) = ... → Slot: ..... (Collisions: .....)
- Key 70: h_0 = 70 mod 19 = ... → Collision at Slot: ..... → f(1) = ... → Slot: ..... (Collisions: .....)
- Key 85: h_0 = 85 mod 19 = ... → Collision 1: ..... → Collision 2: ..... → Slot: ..... (Collisions: .....)
- Key 102: h_0 = 102 mod 19 = ... → Collision 1: ..... → Collision 2: ..... → Slot: ..... (Collisions: .....)
- Key 123: h_0 = 123 mod 19 = ... → Collision 1: ..... → Collision 2: ..... → Slot: ..... (Collisions: .....)
- Key 143: h_0 = 143 mod 19 = ... → Slot: ..... (Collisions: .....)
- Key 155: h_0 = 155 mod 19 = ... → Collision at Slot: ..... → f(1) = ... → Slot: ..... (Collisions: .....)

1.2 Maximum Collisions: Which key(s) experienced the highest number of collisions? State the key(s), the number of collisions, and the specific slot indices where collisions occurred. **[2 Marks]**
Answer: Key(s): ..... | Max Collisions: ..... | Collided at slots: .....

1.3 Total Collision Count: What is the total cumulative number of collisions and resolution steps across all 8 insertions? **[2 Marks]**
Answer: Total Collisions = ...

1.4 Rehashing Threshold: After inserting all 8 keys, how many additional keys must be inserted into the table to trigger Rehashing? (Show calculation step). **[2 Marks]**
Calculation & Answer: .....

Question 2: Binary Min-Heap Operations & Array Representation (10 Marks)

A Binary Min-Heap is represented in a 1-dimensional array using 1-based indexing (where Index 0 is unused). The fundamental array indexing properties are:

• Left Child(i) = 2i • Right Child(i) = 2i + 1 • Parent(i) = floor(i / 2)

You are given the following sequence of 12 integer keys:

64, 14, 24, 1, 20, 2, 5, 10, 6, 8, 15, 20

Perform the operations sequentially from 2.1 to 2.4 and record the final array states in the 4 vertical columns below.

Instructor's Strict Warning: Perform all operations strictly adhering to the algorithmic Percolate-Up and Percolate-Down procedures taught in class. Do not guess the final tree layout by intuition. Grading strictly follows program execution traces.

Index	2.1 Insert 12 Keys [1 Mark]	2.2 deleteMin() 1 [2 Marks]	2.3 Insert key 3 [2 Marks]	2.4 deleteMin() 2 [3 Marks]
[0]	-	-	-	-
[1]				
[2]				
[3]				
[4]				
[5]				
[6]				
[7]				
[8]				
[9]				
[10]				
[11]				
[12]		-		-

2.5 Heap Properties Verification: [2 Marks]

State the two fundamental properties that every Binary Heap must satisfy:

1. Structure Property:

.....

2. Heap-Order Property:

.....

Question 3: Insertion Sort Tracing & Position Moves (10 Marks)

Recall the standard insertionSort algorithm from Chapter 7. A Position Move (or Data Shift) is defined as each shift assignment a[j] = a[j - 1] executed inside the inner loop.

3.1 Worst-Case Analysis: [4 Marks]
Specify an initial input array of 12 integers that represents the exact Worst-Case Scenario for insertion sort. Write the input array in the first table, and calculate the Total Position Moves upon completion.
Input Array (Worst Case):

Index	0	1	2	3	4	5	6	7	8	9	10	11
Value												

Formula & Calculation for Total Position Moves:

.....

Total Position Moves = ... moves.

3.2 Pass Tracing for Input Array: [6 Marks]
Given the initial array of 12 numbers: [61, 52, 12, 24, 8, 35, 32, 5, 71, 68, 18, 2]
Write the array content and count the position moves occurred specifically at Pass p = 3 and Pass p = 6:

Pass	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]	[10]	[11]	Position Moves in Pass
Initial	61	52	12	24	8	35	32	5	71	68	18	2	-
p = 3													
p = 6													

Question 4: Selection Sort & Bubble Sort Tracing (10 Marks)

Trace the execution of Selection Sort and Bubble Sort on the given arrays as instructed below.

4.1 Selection Sort Tracing: [5 Marks]
Given the array [64, 25, 12, 22, 11]. Trace the array contents after each pass. Record the index of the minimum element found and the swap operation performed:

Pass	Current Array State	Min Element Found (Index)	Swap Action
Initial	[64, 25, 12, 22, 11]	-	-
Pass 1			
Pass 2			
Pass 3			
Pass 4			

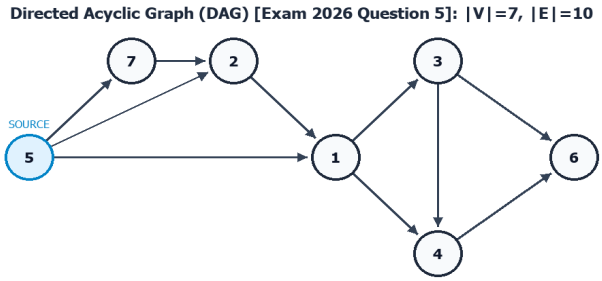
4.2 Bubble Sort Tracing & Early Termination: [5 Marks]
Given the array [54, 26, 93, 17, 77]. Trace each pass, count comparisons and swaps. Does the algorithm terminate early? If so, after which pass?

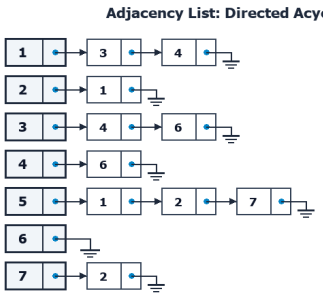
Pass	Array State after Pass	Comparisons in Pass	Swaps in Pass	Swapped (T/F)
Initial	[54, 26, 93, 17, 77]	-	-	-
Pass 1				
Pass 2				
Pass 3				
Pass 4				

Total Comparisons: ..... | Total Swaps: ..... | Early Termination occurred at Pass: .....

Question 5: Topological Sort in Directed Acyclic Graph (DAG) (10 Marks)

A Directed Acyclic Graph (DAG) with 7 vertices (1 to 7) is defined by the graph structure and its linked Adjacency List below:





5.1 Indegree Array Tracing: [5 Marks]

Fill in the Indegree Array below. Show the initial in-degree of each vertex, and show the strikethrough/update values after each round as vertices are dequeued and their incident edges removed:

Vertex (v)	Initial	Round 1	Round 2	Round 3	Round 4	Round 5	Round 6	Round 7
1								
2								
3								
4								
5								
6								
7								
Dequeue (v)								

5.2 Circular Queue Simulation: [2 Marks]

Given a Queue of capacity 7 (Indices 0 to 6) implementing the standard circular queue functions enqueue and dequeue from Chapter 3.

Draw the final contents of the Queue array upon algorithm completion, and explicitly state where front and back point to:

Slot Index	0	1	2	3	4	5	6
Queue Value							

front points to index: ..... **back points to index: .....**

5.3 Final Topological Ordering: [3 Marks]

Write the resulting Topological Ordering sequence produced by the algorithm:

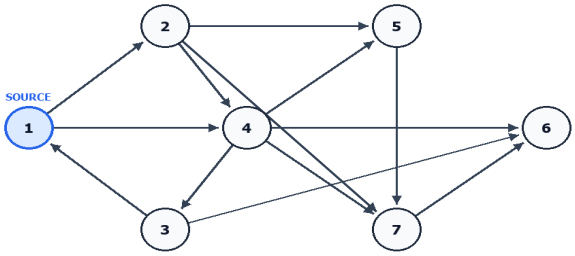
Answer:

.....

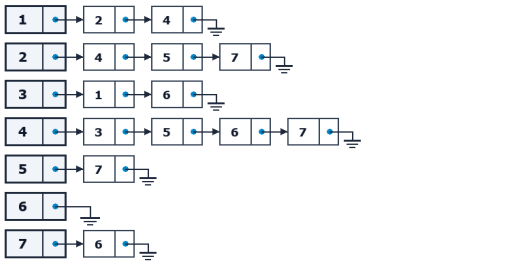
Question 6: Unweighted Shortest Path Algorithm (10 Marks)

A directed unweighted graph with 7 vertices (1 to 7) is defined by the graph structure and its linked Adjacency List below (Source Vertex = 1):

Directed Graph G = (V, E) [Exam 2026 Question 6]: Source Vertex = 1



Adjacency List: Directed Graph (Source = 1, 7 Vertices)



6.1 Execution Table for Shortest Path (Starting Vertex = 1): [5 Marks]

Execute the BFS-based unweighted shortest path algorithm. Fill in the final processing array table containing Known (T/F or 1/0), Distance (d_v), and Path (p_v) for all 7 vertices. Show initial values (d_v = ∞ or 999, p_v = 0) and the final completed values when Known is True for all vertices:

Vertex (v)	Known (T / F)	Distance (d_v)	Previous Path (p_v)	Shortest Route from Source (1)
1				
2				
3				
4				
5				
6				
7				

6.2 Circular Queue State upon Completion: [2 Marks]

Given a Queue of capacity 7 (Slots 0 to 6) used to process vertices in the algorithm. Draw the final contents of the Queue array and specify the pointer positions after the algorithm finishes:

Slot Index	0	1	2	3	4	5	6
Queue Value							

front points to index: ..... **back points to index: .....**

6.3 Shortest Path Reconstruction & Zero-Mark Warning: [3 Marks]

Professor's Rule: Path sequences MUST be written as comma-separated sequences (e.g. 1, 4, 5). DO NOT write arrows (1 → 4 → 5) which receives 0 marks. Path length is measured in number of EDGES.

- Shortest Path to Vertex 5: Sequence = ... | Length = ... edges
- Shortest Path to Vertex 7: Sequence = ... | Length = ... edges

Question 7: Advanced Theory, Structural Properties & Calculations (10 Marks)

Answer the following short theory and mathematical questions concisely. All numerical calculations must be computed as exact integers or percentages (do NOT leave answers in exponent or algebraic forms).

7.1 Structural Properties (Data Structures Classification): **[3 Marks]**

(a) Which fundamental data structure possesses exactly two formal properties? State the structure and define both properties.

Data Structure: .....

1. .....

2. .....

(b) Which tree data structure possesses exactly three formal properties? State the structure and list the properties.

Data Structure: .....

Properties:

.....

7.2 Hash Function Theoretical Properties & Realistic Limitation: **[2 Marks]**

State the 3 ideal properties of a theoretical hash function:

1. .....

2. .....

3. .....

Which of these 3 properties is impossible to achieve in the physical real world, and explain the two fundamental reasons why?

- Impossible Property: .....
- Reason 1:

.....

- Reason 2:

.....

7.3 Graph Memory Waste Percentage (Sparse Graph Analysis): **[2 Marks]**

A directed graph has $|V| = 7$ vertices and $|E| = 12$ edges.

- What is the total number of cells in its Adjacency Matrix? .....
- What percentage of matrix cells contain 1 (active edges)? .....
- What percentage of matrix cells are 0 (wasted unused space)? .....
- How many cells are required if represented as an Adjacency List ($|V| + |E|$)? .....

7.4 Complete Graph Edge Calculation: **[1 Mark]**

A complete undirected graph K_{16} has $|V| = 16$ vertices. How many edges are in this graph? (Must answer as an exact integer, do NOT leave as formula).

Calculation & Answer:

.....

7.5 Binary Tree Node Bounds: **[1 Mark]**

A Complete Binary Tree has height $h = 16$ (assuming Root is at height 0).

- Minimum number of nodes: ..... (Must be exact integer, not 2^{16})
- Maximum number of nodes: ..... (Must be exact integer, not $2^{17} - 1$)

7.6 Binary Heap 1-Based Array Index Formulas: **[1 Mark]**

For a node located at index i in a 1-based array implementation of a Binary Heap, state the exact formula for:

- Left Child Index: ..... • Right Child Index: ..... • Parent Index: .....

***** END OF EXAMINATION PAPER — WISHING YOU ALL THE BEST! *****